

Key–Value Cache Retrieval Is No Longer Cheap in Visual Autoregressive Models

Jain Koo, Enhyeok Jang, Won Woo Ro
School of Electrical and Electronic Engineering
Yonsei University
Seoul, Republic of Korea
{jain.koo, enhyeok.jang, wro}@yonsei.ac.kr

Abstract—Visual autoregressive (VAR) models generate images via next-resolution prediction, producing an entire token map at each decoding stage. Recent works adopt key–value (KV) cache retrieval to reduce memory traffic by selectively loading critical KV entries. These approaches typically assume that criticality estimation incurs only negligible overhead, as in large language models (LLMs) where each step involves a single query token. Unfortunately, that scoring is not so lightweight on VAR, as it processes multiple tokens simultaneously at each generation stage, due to its multiple token generation process. In this work, we analyze the latency characteristics of KV cache retrieval in the Infinity-8B model to understand its impact on inference performance. Our analysis shows that attention dominates the overall runtime in later stages, and that KV cache loading accounts for the majority of the attention latency.

I. INTRODUCTION

Visual autoregressive (VAR) models [1] have recently emerged as an effective approach for high-resolution image generation. Similar to autoregressive transformers used in large language models (LLMs) [3], VAR inference can rely on attention operations with growing key–value (KV) caches.

However, there is a fundamental difference between conventional LLMs and VAR models. Unlike conventional LLM models that generate a single token at each decoding step, VAR produces an entire token map at each stage, progressively refining visual representations across scales. This paradigm significantly reduces the number of autoregressive iterations required for high-resolution synthesis. However, it fundamentally changes the inference characteristics of Transformer-based generation compared to conventional LLMs and AR models. In particular, the multi-token generation behavior of VAR introduces substantial computational and memory challenges during inference. Because each stage generates an entire token map rather than a single token, the sequence length grows explosively across stages, causing the KV cache size to increase far more rapidly than in LLMs, where it grows only linearly with decoding steps.

Because the KV cache grows over generations, accessing cached keys and values can become a major source of latency during inference. Recent KV cache retrieval approaches assume that importance estimation incurs negligible overhead, as each decoding step involves only a single query token. However, this assumption does not hold in VAR, where multiple query tokens are processed simultaneously. As a result, the

cost of importance estimation may become non-negligible and even dominate the attention pipeline.

This work analyzes the latency characteristics of KV cache retrieval in Infinity-8B [5] to understand its impact.

II. BACKGROUND AND MOTIVATION

A. Visual Autoregressive Modeling

Autoregressive models [3] for image and video generation have recently attracted significant attention as promising alternatives to diffusion models because they exhibit strong scalability and generality, similar to language autoregressive models such as LLMs [2], [3]. However, conventional AR models rely on next-token prediction, which inevitably leads to a large number of decoding steps. This issue becomes particularly severe for high-resolution image synthesis, where the number of tokens grows quadratically with spatial resolution, resulting in many autoregressive iterations and high end-to-end latency.

To address this limitation, Visual Autoregressive Models (VAR [1], [5]) shift the generation paradigm from next-token prediction to next-resolution prediction. Motivated by the coarse-to-fine nature of visual content, VAR progressively predicts higher-resolution visual representations conditioned on previously generated lower-resolution ones, substantially reducing the number of autoregressive iterations required for high-resolution synthesis. More specifically, VAR represents an image as a sequence of multi-scale token maps, $\mathbf{R} = (\mathbf{r}_1, \dots, \mathbf{r}_K)$, where the spatial resolution increases with the scale index k , and $\mathbf{r}_K$ corresponds to the highest-resolution stage. During inference, when predicting $\mathbf{r}_k \in [V]^{H_k \times W_k}$, VAR generates all $H_k \times W_k$ tokens in parallel within the k -th stage, enabling the simultaneous generation of thousands of tokens (e.g., 4096 tokens at the 12^{th} stage in Infinity-8B), leading to rapid KV cache growth as shown in Figure 1. This multi-token generation property fundamentally distinguishes VAR from conventional AR models such as LLMs, where each decoding step typically predicts only a single token.

Although this design substantially reduces the number of autoregressive iterations, it also introduces unique performance challenges during inference. Similar to other AR models, VAR is built upon a Transformer architecture and operates on extremely long sequences for high-resolution generation, leading to substantial attention latency. Furthermore, unlike conventional AR models that employ causal masking, VAR

does not use a causal mask within each token map, resulting in a higher computation cost of attention for the same sequence length.

These challenges also extend to memory efficiency. Although VAR adopts KV caching to improve inference throughput, its multi-token generation behavior leads to fundamentally different KV cache scaling characteristics. While the KV cache size in conventional AR models grows linearly with the number of decoding steps, VAR generates multiple tokens corresponding to an entire resolution at each step, causing the KV cache size to grow much more rapidly and resulting in significantly larger memory requirements during inference.

B. KV Cache Retrieval Systems

To address the computation and memory inefficiencies caused by high attention computation cost and large KV cache footprints, recent works have primarily focused on reducing the effective KV cache size, as this simultaneously lowers memory overhead and attention computation cost. A straightforward approach is to compress the KV cache so that it fits within limited GPU memory, typically through techniques such as KV cache eviction and quantization. However, both approaches incur irreversible information loss, which can lead to significant accuracy degradation, especially in long-sequence scenarios.

An alternative paradigm is *KV cache retrieval* [4], which aims to preserve accuracy without permanently discarding information. Rather than compressing the cache, retrieval-based methods store the full KV cache in larger memory, such as CPU DRAM or storage, and selectively load only critical KV entries back to the GPU. In general, this process involves partitioning KV entries, estimating their importance, and retrieving a small subset for GPU execution. By restricting attention to this subset, these methods reduce both computation and memory overhead while largely preserving generation quality and enabling scaling beyond GPU memory capacity.

However, this approach introduces a new bottleneck due to the limited PCIe bandwidth between CPU and GPU, as on-demand KV retrieval can expose data movement latency on the critical path and limit inference performance. To mitigate this bottleneck, prior works further incorporate KV cache prefetching to overlap PCIe transfers with GPU computation, hiding data movement latency from the critical path [4].

To make KV cache retrieval practical under limited PCIe bandwidth, two key design objectives must be satisfied. First, the criticality estimation stage, which determines the importance of KV entries, must be highly efficient. Since this stage is introduced in addition to the original inference pipeline, it should incur minimal overhead so that selected KV entries can be prefetched to the GPU in time, preventing stalls in subsequent attention computation. Second, the criticality estimation must be sufficiently accurate. Selecting unimportant KV entries can lead to generation quality degradation. Moreover, inaccurate estimation often necessitates fetching a larger number of KV entries to maintain quality, which in

turn increases PCIe traffic and diminishes the effectiveness of latency hiding through prefetching.

Prior works have successfully satisfied these requirements in conventional AR models. However, their effectiveness may not directly carry over to VAR due to its multi-token decoding behavior. Unlike conventional AR models, which generate a single query token at each decoding step, VAR generates multiple queries simultaneously through next-resolution prediction. This difference may introduce additional challenges for both the efficiency and the accuracy of criticality estimation in KV retrieval systems.

C. Visual Autoregressive and Their Workload Characteristics

VARs [1], [5] generate images progressively by predicting groups of tokens at multiple spatial resolutions, where each stage corresponds to a spatial map with increasing resolution. If the spatial resolution at stage k is $H_k \times W_k$, the query tensor contains $Q_k = H_k W_k$ tokens simultaneously. As generations proceed, new tokens are appended to the KV cache, thereby growing the set of cached keys and values across the stages.

Attention computation at stage k involves a query matrix $Q_k \in \mathbb{R}^{Q_k \times d}$ interacting with a large set of cached keys. The attention scoring becomes $S = QK^\top$, which corresponds to a generalized matrix-matrix multiplication (GEMM). Compared to GEMV for LLM decoding, GEMM could impose a greater operational burden and memory traffic during attention.

D. Implications of Cache Retrievals for Visual Autoregressive

The efficiency of KV cache retrievals relies on the assumption that importance estimation is negligible. This assumption holds for typical LLM decoding, where only a single query token is processed at each decoding step. In this case, importance estimation reduces to multiplying a single query vector by the keys. Unfortunately, this assumption may not hold for visual autoregressive models. VAR models generate multiple tokens simultaneously at each stage, resulting in a query matrix containing multiple query vectors. Due to this, importance estimation requires computing importance scores between the query vectors and cached keys.

This computation corresponds to a GEMM rather than a GEMV, as discussed in Section II-C. Such a difference can increase the computational cost of importance estimation and may introduce non-negligible overhead in KV cache retrieval. Understanding how this workload characteristic affects KV cache retrieval overheads is therefore essential for the VARs.

III. KV CACHE RETRIEVAL OVERHEAD ANALYSIS

A. Experimental Setup and Environment for Latency Analysis

For accurate modeling, we estimate data transfer latency using an NVIDIA RTX A6000 GPU with 48 GB of device memory and AMD EPYC™ Model 7543, with DDR4-based CPU memory. The operating system (OS) is Ubuntu 20.04.2 LTS. The Infinity-8B model [1], [5] is executed on the GPUs to measure a latency breakdown across various stage indices.

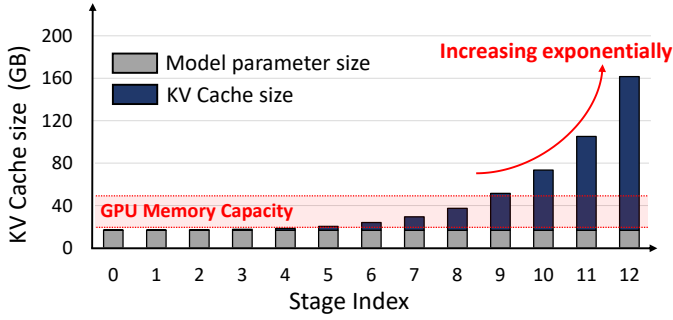


Fig. 1. KV cache size analysis in Infinity-8B, at batch size 4. Unlike LLMs, the KV cache size in VAR grows exponentially during inference.

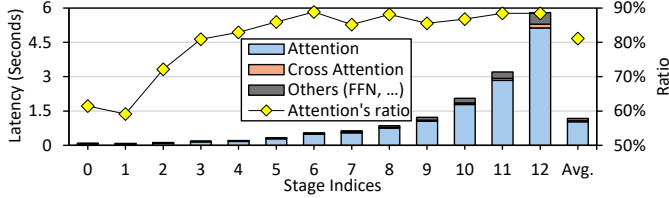


Fig. 2. Stage index-wise latency breakdown of the Infinity-8B model [1], [5]. Each bar shows the latency contributions of attention, cross attention, and other operations (e.g., feed-forward networks) across generation stage indices. The line with markers indicates the ratio of attention latency to the total stage latency. As the stage index increases, overall latency increases near-exponentially, and thereby attention gradually dominates execution time.

B. End-to-End Stage Index-Wise Latency Breakdown Analysis

Figure 2 presents the end-to-end latency breakdown of the Infinity-8B model [1], [5] across various generation stage indices. We observe that the overall latency increases rapidly as the stage index grows. For example, the total latency increases from approximately 0.09 seconds at stage 0 to about 5.8 seconds at stage 12. This increase is primarily driven by the growing cost of attention computation. In particular, attention dominates the execution time in most stages. Across all stages, attention accounts for approximately 81% of the total latency on average. In later stages, the dominance becomes more pronounced, where attention consistently contributes more than 85% of the total latency. At stage 12, attention alone accounts for about 5.13 seconds out of the total 5.80 seconds of execution time. In contrast, cross attention and other operations contribute only a small portion of the total latency throughout the generation process. These results indicate that attention computation constitutes the primary workload in the Infinity-8B model, particularly in later stages. Given that the attention workload dominates the overall execution time, we next analyze the internal workload of the attention computation to understand the overhead associated with KV cache retrieval.

C. Attention Workload Breakdown Analysis at Stage Index 11

To further understand attention latency, we perform a more detailed breakdown of the 11th-generation stage in the Infinity-8B model. Table I presents the latency breakdown of the attention workload at stage 11. The results show that KV cache loading dominates the attention latency. Specifically, KV cache

TABLE I
ATTENTION LATENCY BREAKDOWN IN INFINITY-8B (STAGE INDEX = 11)

Component	Latency (s)	Ratio (%)
KV Cache Loading	1.543	57.1
Rotary Embedding	0.211	7.8
Linear Projection	0.196	7.3
Attention Computation	0.469	17.3
Others	0.284	10.5
Total	2.703	100

loading alone accounts for 1.543 seconds, which corresponds to 57.1% of the total attention latency. In contrast, the actual attention computation contributes a much smaller portion. The matrix multiplication involved in attention score computation accounts for only 0.469 seconds (17.3%). Other components, including rotary positional embedding and linear projection, contribute 7.8% and 7.3%, respectively. These results indicate that the primary bottleneck of attention computation in the Infinity-8B model is not the arithmetic computation itself, but the overhead associated with KV cache retrieval and loading.

IV. CONCLUSION

This work investigates the overhead of KV cache retrieval in VAR models. Unlike conventional autoregressive models, VAR generates multiple tokens simultaneously at each stage, which alters the cost structure of criticality estimation. Through latency analysis of Infinity-8B, we show that attention dominates the overall runtime in later stages and that KV cache loading constitutes the majority of the attention latency. These findings indicate that KV retrieval in VAR introduces non-negligible overhead, in contrast to prior assumptions derived from LLM-based decoding. Our results highlight the need to reconsider KV retrieval design in the context of multi-token generation and provide insights for building efficient VAR inference.

ACKNOWLEDGMENTS

This work was supported by Samsung Electronics Co., Ltd.(IO251210-14271-01), and Institute of Information & Communication Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2025-0-00441, Memory-Centric Architecture Using the Reconfigurable PIM Devices). Won Woo Ro is the corresponding author.

REFERENCES

- [1] K. Tian, Y. Jiang, Z. Yuan, B. Peng, and L. Wang, "Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction," arXiv preprint arXiv:2404.02905, 2024.
- [2] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast autoregressive transformers with linear attention," in International Conference on Machine Learning, pp. 5156–5165, 2020.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is All you Need," in Advances in Neural Information Processing Systems, vol. 30, Curran Associates, Inc., 2017.
- [4] Kim, Donghyuk and Yang, Sejeong and Shin, Wonjin and Kim, Joo-Young "V-rex: Real-time streaming video llm acceleration via dynamic kv cache retrieval," arXiv preprint arXiv:2512.12284, 2025.

- [5] J. Han, J. Liu, Y. Jiang, B. Yan, Y. Zhang, Z. Yuan, B. Peng, and X. Liu, “Infinity: Scaling Bitwise AutoRegressive Modeling for High-Resolution Image Synthesis,” arXiv preprint arXiv:2412.04431, 2024.